

# DBMS Unit 2 - Relational Data Model

## UNIT - II: RELATIONAL DATA MODEL

MSBTE K-Scheme | Diploma CO | SY - Database Management System

---

### 2.1 RELATIONAL STRUCTURE

The relational model represents data in the form of **tables**. Below are the fundamental building blocks of this structure.

#### 1. Tables (Relations)

A **table (relation)** is a two-dimensional structure used to store data in the form of rows and columns. Each table has a unique name in the database.

- A relation is formally a **set of tuples**.
- Order of rows and columns does not matter in a true relation.
- Every table must have at least one column that can uniquely identify each row.

**Example - STUDENT table (a relation):**

| Roll_No | Name  | Branch |
|---------|-------|--------|
| 1       | Aditi | CO     |
| 2       | Rahul | IF     |

#### 2. Rows (Tuples)

A **row**, also called a **tuple**, represents a single record in a table — i.e., one complete set of related data values.

- Each row must be unique in a relation (no two identical rows are allowed).
- The number of rows in a relation is called its **cardinality**.

Example: (1, "Aditi", "CO") is one tuple in the STUDENT table.

#### 3. Domains

A **domain** is the set of all possible/permissible values that an attribute (column) can take.

- Domain defines the data type and range/constraints of an attribute.
- Example: The domain of "Marks" could be integers from 0 to 100. The domain of "Gender" could be {Male, Female, Other}.

#### 4. Attributes

An **attribute** is a column of a table that represents a property or characteristic of the entity.

- Each attribute has a name and is associated with a domain.
- The number of attributes in a relation is called its **degree**.

Example: In STUDENT(Roll\_No, Name, Branch) — Roll\_No, Name, and Branch are attributes; degree = 3.

## 5. Entities

An **entity** is a real-world object or concept that has an independent existence and can be distinctly identified.

- Entities can be **tangible** (Student, Employee, Car) or **intangible** (Bank Account, Course, Project).
  - An **entity set** is a collection of similar entities. Example: all students form the “STUDENT” entity set.
  - Each entity is represented as a row in a relational table once implemented.
- 

## 2.2 KEYS

Keys are attributes (or a set of attributes) used to uniquely identify tuples in a relation and to establish relationships between tables.

### 1. Super Key

A **super key** is a set of one or more attributes that, taken collectively, can uniquely identify a tuple in a relation.

- A super key may contain extra/redundant attributes that are not necessary for unique identification.
- Every relation has at least one super key (the set of all attributes).

**Example:** In STUDENT(Roll\_No, Name, Email), possible super keys are: {Roll\_No}, {Email}, {Roll\_No, Name}, {Roll\_No, Name, Email}.

### 2. Candidate Key

A **candidate key** is a **minimal super key** — i.e., a super key with no redundant/unnecessary attributes. Removing any attribute from a candidate key would make it lose its uniqueness property.

- A relation can have **multiple candidate keys**.
- Each candidate key can independently and uniquely identify a tuple.

**Example:** In STUDENT(Roll\_No, Name, Email), {Roll\_No} and {Email} are both candidate keys (assuming both are unique).

### 3. Primary Key

A **primary key** is the candidate key **selected by the database designer** to uniquely identify tuples in a relation.

**Properties of Primary Key:** - Must contain **unique** values for every row. - **Cannot contain NULL** values (Entity Integrity Rule). - Only **one primary key** is allowed per table. - Once chosen, the remaining candidate keys are called **alternate keys**.

**Example:** In STUDENT(Roll\_No, Name, Email), Roll\_No is chosen as the Primary Key.

## 4. Foreign Key

A **foreign key** is an attribute (or set of attributes) in one relation (child/referencing table) that refers to the **primary key** of another relation (parent/referenced table).

- Establishes a **link/relationship** between two tables.
- A foreign key can accept **NULL** values (unless otherwise restricted) and **duplicate** values.
- Used to enforce **referential integrity**.

### Example:

STUDENT table: | Roll\_No (PK) | Name | Dept\_ID | |—|—|—|

DEPARTMENT table: | Dept\_ID (PK) | Dept\_Name | |—|—|

Here, Dept\_ID in STUDENT is a **foreign key** referencing Dept\_ID (primary key) in DEPARTMENT.

### Summary Table of Keys:

| Key           | Uniqueness      | Minimality      | NULL allowed | Count per table |
|---------------|-----------------|-----------------|--------------|-----------------|
| Super Key     | Yes             | Not necessarily | —            | Multiple        |
| Candidate Key | Yes             | Yes             | No           | Multiple        |
| Primary Key   | Yes             | Yes             | No           | Only One        |
| Foreign Key   | No (can repeat) | —               | Yes          | Multiple        |

## 2.3 DATA CONSTRAINTS

Constraints are rules enforced on data columns to maintain the **accuracy, consistency, and reliability** of data in the database.

### 1. Domain Constraints

A **domain constraint** specifies that the value of each attribute must be an **atomic value** from the specified domain of that attribute.

- Ensures data type correctness (e.g., Age must be an integer, Name must be text).
- Ensures value range correctness (e.g., Marks must be between 0 and 100).
- Prevents insertion of invalid data types or out-of-range values.

**Example:** If the domain of “Age” is defined as integers between 0–100, then entering “Twenty” or “-5” violates the domain constraint.

### 2. Referential Integrity Constraint

The **referential integrity constraint** is specified between two relations and is used to maintain consistency among tuples in the two relations connected via a foreign key.

**Rule:** The value of a foreign key in the referencing (child) table must either: - Match a value of the primary key in the referenced (parent) table, **or** - Be **NULL**.

**Purpose:** - Prevents insertion of a foreign key value that does not exist in the parent table. - Prevents deletion of a parent record if it is still referenced by a child record (unless cascading rules are defined).

**Example:** If DEPARTMENT table does not have Dept\_ID = 5, then no row can be inserted into STUDENT with Dept\_ID = 5.

**Other important constraints (related concepts):** - **Entity**

**Integrity Constraint:** No attribute of a primary key can be NULL. -

**Key Constraint:** No two tuples can have the same value for the primary key.

---

## 2.4 ENTITY RELATIONSHIP (ER) MODEL

The **ER Model** is a high-level conceptual data model used to design a database by representing data as entities, attributes, and relationships before implementation.

### 1. Strong Entity Set

A **strong entity set** is an entity set that has a **primary key** of its own and does **not depend** on any other entity set for its existence.

- Represented by a **single rectangle** in ER diagrams.
- Example: STUDENT, EMPLOYEE (each has its own unique identifier like Roll\_No or Emp\_ID).

### 2. Weak Entity Set

A **weak entity set** is an entity set that does **not have a primary key of its own** and depends on a strong (owner/identifying) entity set for its identification.

- Uses a **partial key (discriminator)** combined with the primary key of the owner entity to form a unique identifier.
- Represented by a **double rectangle** in ER diagrams.
- Connected to the owner entity through an **identifying relationship**, shown using a **double diamond**.

**Example:** DEPENDENT entity (of an employee) cannot exist without the EMPLOYEE entity. "Dependent Name" may be the partial key, combined with Emp\_ID to uniquely identify a dependent.

### 3. Types of Attributes

| Attribute Type                   | Description                            | Example                          |
|----------------------------------|----------------------------------------|----------------------------------|
| <b>Simple (Atomic) Attribute</b> | Cannot be divided further              | Roll_No                          |
| <b>Composite Attribute</b>       | Can be divided into sub-parts          | Name (First_Name + Last_Name)    |
| <b>Single-valued Attribute</b>   | Has only one value for an entity       | Date_of_Birth                    |
| <b>Multi-valued Attribute</b>    | Can have multiple values for an entity | Phone_Numbers                    |
| <b>Derived Attribute</b>         | Value derived from other attributes    | Age (derived from Date_of_Birth) |
| <b>Key Attribute</b>             | Uniquely identifies each entity        | Roll_No, Emp_ID                  |

### 4. Symbols Used in ER Diagram

---

| Symbol           | Shape   | Represents                                                     |
|------------------|---------|----------------------------------------------------------------|
| Rectangle        | □       | Entity (Strong Entity Set)                                     |
| Double Rectangle | □□      | Weak Entity Set                                                |
| Ellipse/Oval     | ○       | Attribute                                                      |
| Double Ellipse   | ○○      | Multi-valued Attribute                                         |
| Dashed Ellipse   | - - -   | Derived Attribute                                              |
| Diamond          | ◇       | Relationship                                                   |
| Double Diamond   | ◇◇      | Identifying Relationship (for weak entity)                     |
| Line             | —       | Connects attributes to entities, and entities to relationships |
| Underline        | Roll_No | Denotes Key/Primary Key attribute                              |

## 5. ER Diagrams

An **ER diagram** is a visual/graphical representation of entities, their attributes, and the relationships among them, used as a blueprint for designing the database schema.

**Steps to draw an ER diagram:** 1. Identify entities (e.g., STUDENT, COURSE) 2. Identify attributes of each entity (e.g., Roll\_No, Name for STUDENT) 3. Identify relationships between entities (e.g., STUDENT “enrolls in” COURSE) 4. Identify the **cardinality/degree** of relationships: - **One-to-One (1:1)** - **One-to-Many (1:M)** - **Many-to-Many (M:N)** 5. Represent using proper symbols (rectangles, ellipses, diamonds)

### Example ER Diagram (conceptual):

```

[STUDENT]-----<enrolls in>-----[COURSE]
Roll_No           (M:N)           Course_ID
Name              Course_Name
Branch

```

Here STUDENT and COURSE are entities, “enrolls in” is the relationship, and the relationship is Many-to-Many since one student can enroll in many courses and one course can have many students.

## 2.5 NORMALIZATION

**Normalization** is the process of organizing data in a database to **reduce data redundancy** and **improve data integrity** by decomposing large tables into smaller, well-structured tables based on rules called **normal forms**.

### 1. Functional Dependency (FD)

A **functional dependency** is a constraint between two sets of attributes in a relation, denoted as  $X \rightarrow Y$  (read as “X determines Y”), meaning: for any two tuples, if the values of X are the same, then the values of Y must also be the same.

- **X** is called the **determinant**.
- **Y** is called the **dependent**.

**Example:** In STUDENT(Roll\_No, Name, Branch), Roll\_No  $\rightarrow$  Name means Roll\_No uniquely determines Name.

**Types of Functional Dependency:**

- **Full Functional Dependency:** Y is fully dependent on the entire composite key X, not on any subset of X.
- **Partial Functional Dependency:** Y depends on only a part (subset) of a composite candidate key.
- **Transitive Dependency:** If  $X \rightarrow Y$  and  $Y \rightarrow Z$ , then  $X \rightarrow Z$  (Z depends indirectly on X through Y).

## 2. Normal Forms

Normalization is carried out in progressive stages called **normal forms**. Each higher normal form depends on satisfying the previous one.

---

### First Normal Form (1NF)

A relation is in **1NF** if:

- Every attribute contains only **atomic (indivisible)** values.
- There are **no repeating groups** or multi-valued attributes/arrays in a single column.

**Example (Not in 1NF):**

---

| Roll_No | Name  | Phone_Numbers          |
|---------|-------|------------------------|
| 1       | Aditi | 9876543210, 9123456780 |

---

**Converted to 1NF:**

---

| Roll_No | Name  | Phone_Number |
|---------|-------|--------------|
| 1       | Aditi | 9876543210   |
| 1       | Aditi | 9123456780   |

---

### Second Normal Form (2NF)

A relation is in **2NF** if:

- It is already in **1NF**, and
- It has **no partial dependency** — i.e., no non-prime attribute (attribute not part of any candidate key) is dependent on only a **part** of a composite primary key.

*(2NF is only relevant when the primary key is composite — made of more than one attribute.)*

**Example (Not in 2NF):**

---

| Roll_No | Course_ID | Course_Name | Marks |
|---------|-----------|-------------|-------|
|---------|-----------|-------------|-------|

Here, composite key = (Roll\_No, Course\_ID). But Course\_Name depends only on Course\_ID (a part of the key) — this is a **partial dependency**.

**Converted to 2NF (split into two tables):**

STUDENT\_COURSE: | Roll\_No | Course\_ID | Marks | |—|—|—|

COURSE: | Course\_ID | Course\_Name | |—|—|

---

### Third Normal Form (3NF)

A relation is in **3NF** if:

- It is already in **2NF**, and
- It has **no transitive dependency** — i.e., no non-prime attribute depends on another non-prime attribute (all non-key attributes must depend directly on the primary key only).

**Example (Not in 3NF):**

---

**Roll\_No   Name   Dept\_ID   Dept\_Name**

Here, Roll\_No → Dept\_ID and Dept\_ID → Dept\_Name, so Roll\_No → Dept\_Name is a **transitive dependency** (through Dept\_ID).

**Converted to 3NF (split into two tables):**

STUDENT: | Roll\_No | Name | Dept\_ID | |—|—|—|

DEPARTMENT: | Dept\_ID | Dept\_Name | |—|—|

---

## Summary Table of Normal Forms

| Normal Form | Condition                                 | Removes                                          |
|-------------|-------------------------------------------|--------------------------------------------------|
| <b>1NF</b>  | All attribute values must be atomic       | Repeating groups / multi-valued attributes       |
| <b>2NF</b>  | Must be in 1NF + no partial dependency    | Partial dependency on composite key              |
| <b>3NF</b>  | Must be in 2NF + no transitive dependency | Transitive dependency between non-key attributes |

**Advantages of Normalization:** - Reduces data redundancy - Improves data consistency and integrity - Avoids insertion, update, and deletion anomalies - Makes the database more flexible and easier to maintain

---

## Quick Revision Points

- **Relation** = table; **Tuple** = row; **Attribute** = column; **Domain** = allowed values
- **Degree** = number of columns; **Cardinality** = number of rows
- **Super Key** ⊇ **Candidate Key** ⊇ **Primary Key**; **Foreign Key** links two tables
- **Domain Constraint** = valid values; **Referential Integrity** = FK must match PK or be NULL
- **Strong Entity** = has own key; **Weak Entity** = depends on owner, uses partial key
- **ER Diagram symbols:** Rectangle = Entity, Ellipse = Attribute, Diamond = Relationship
- **Functional Dependency:** X → Y means X determines Y
- **1NF** → atomic values | **2NF** → no partial dependency | **3NF** → no transitive dependency